



Reinforcement Learning

P Satya Jayadev

Web Enabled M.Tech,
Indian Institute of Technology Madras

Topic 3

TD Learning Methods



1. Intro to TD Learning
2. 1-step TD Learning
3. On-Policy TD Control: SARSA
4. Off-Policy TD Control: Q-Learning
5. Example: SARSA Vs Q-Learning
6. On-Policy TD Control: Expected SARSA
7. n-Step TD Methods and $TD(\lambda)$

References: Sutton, R. S., Barto, A. G. (2018). Reinforcement Learning: An Introduction. The MIT Press.



Intro to Temporal Difference (TD) Learning

- ▶ “If one had to identify one idea as central and novel to RL, it would undoubtedly be temporal-difference (TD) learning.” - Sutton and Barto
- ▶ Can be used for both episodic as well as continuing tasks



Intro to Temporal Difference (TD) Learning

- ▶ “If one had to identify one idea as central and novel to RL, it would undoubtedly be temporal-difference (TD) learning.” - Sutton and Barto
- ▶ Can be used for both episodic as well as continuing tasks
- ▶ **Basic Approach:** Update the state or action value after one or more steps of MDP (partial episode or bootstrapping)
- ▶ TD learning also comprises of policy evaluation and control



One Step Temporal Difference (TD) Learning

- ▶ Value of a state or state-action pair is updated at each timestep by computing TD error
- ▶ **Recall:** In Incremental MC, error is $(G_t - V(S_t))$



One Step Temporal Difference (TD) Learning

- ▶ Value of a state or state-action pair is updated at each timestep by computing TD error
- ▶ **Recall:** In Incremental MC, error is $(G_t - V(S_t))$
- ▶ In TD error, return G_t is replaced by TD target based on observed reward and next state at $t + 1$: $(R_{t+1} + \gamma V(S_{t+1}))$



One Step Temporal Difference (TD) Learning

- ▶ Value of a state or state-action pair is updated at each timestep by computing TD error
- ▶ **Recall:** In Incremental MC, error is $(G_t - V(S_t))$
- ▶ In TD error, return G_t is replaced by TD target based on observed reward and next state at $t + 1$: $(R_{t+1} + \gamma V(S_{t+1}))$
- ▶ In one step TD, value of a state at time t is updated as:

$$V(S_t) \leftarrow V(S_t) + \alpha[R_{t+1} + \gamma V(S_{t+1}) - V(S_t)]$$

- ▶ TD(0) error: $[R_{t+1} + \gamma V(S_{t+1}) - V(S_t)]$



TD(0) algorithm for Policy Evaluation

- ▶ **Input:** Policy π to be evaluated
- ▶ **Initialise:** $V(s) \forall s \in \mathcal{S}$ and $\alpha \in (0, 1]$
- ▶ **Loop for each timestep t :**
 - Based on state S_t , take action A_t given by policy π and observe R_{t+1} and S_{t+1}
 - Update state value: $V(S_t) \leftarrow V(S_t) + \alpha[R_{t+1} + \gamma V(S_{t+1}) - V(S_t)]$



TD(0) algorithm for Policy Evaluation

- ▶ **Input:** Policy π to be evaluated
- ▶ **Initialise:** $V(s) \forall s \in \mathcal{S}$ and $\alpha \in (0, 1]$
- ▶ **Loop for each timestep t :**
 - Based on state S_t , take action A_t given by policy π and observe R_{t+1} and S_{t+1}
 - Update state value: $V(S_t) \leftarrow V(S_t) + \alpha[R_{t+1} + \gamma V(S_{t+1}) - V(S_t)]$

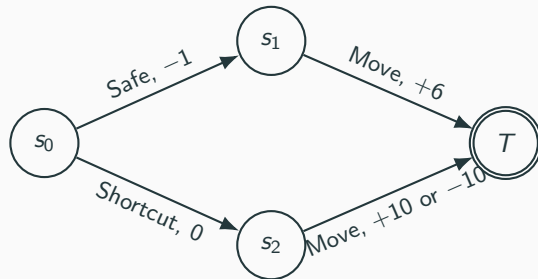
Result: It is proven that following TD(0) with a fixed policy π , $V(s)$ converges to $V_\pi(s)$ when α is constant and small (or) α decreases gradually

Note: There is no result on whether MC or TD methods converge faster but mostly TD is observed to be faster than MC



Example: Robot Choosing Safe Path or Shortcut

- ▶ A robot wants to reach the charging station T .
- ▶ From the start state s_0 , it can choose:
 - a safe path through s_1
 - a risky shortcut through s_2
- ▶ We evaluate a fixed policy using TD(0).



$$\gamma = 0.9, \quad \alpha = 0.5$$



Fixed Policy to be Evaluated

At s_0 :

$$\pi(\text{Safe}|s_0) = 0.8, \quad \pi(\text{Shortcut}|s_0) = 0.2$$

At s_1 and s_2 , there is only one available action:

$$\pi(\text{Move}|s_1) = 1, \quad \pi(\text{Move}|s_2) = 1$$

Initial value estimates:

$$V_0(s_0) = 0, \quad V_0(s_1) = 0, \quad V_0(s_2) = 0, \quad V_0(T) = 0$$

Goal

Estimate $V^\pi(s)$ for the states visited under this policy.



TD(0) Update Reminder

- ▶ At each step, TD(0) updates the value of the current state.
- ▶ The update uses:
 - the reward just received
 - the current estimate of the next state

$$V(S_t) \leftarrow V(S_t) + \alpha [R_{t+1} + \gamma V(S_{t+1}) - V(S_t)]$$

TD Target

$$R_{t+1} + \gamma V(S_{t+1})$$

Key Point

Unlike MC, TD does not wait until the episode ends.



Episode 1: Safe Path is Observed

Suppose the first episode generated by policy π is:

$$s_0 \xrightarrow{\text{Safe}, -1} s_1 \xrightarrow{\text{Move}, +6} T$$

TD Update for s_0 :

$$V_1(s_0) = V_0(s_0) + \alpha(R_{t+1} + \gamma V_0(s_1) - V_0(s_0)) = 0 + 0.5[-1 + 0.9(0) - 0] = -0.5$$

Observation

TD immediately reflects the movement cost -1 , even before the episode ends.



Episode 1: Updating s_1

TD Update for s_1 :

$$V_1(s_1) = V_0(s_1) + \alpha(R_{t+1} + \gamma V_0(T) - V_0(s_1)) = 0 + 0.5[6 + 0.9(0) - 0] = 3$$

After Episode 1:

$$V_1(s_0) = -0.5, \quad V_1(s_1) = 3, \quad V_1(s_2) = 0$$

Observation

s_1 becomes valuable because it leads directly to the charging station.



Episode 2: Shortcut Path is Observed

Suppose the second episode generated by policy π is:

$$s_0 \xrightarrow{\text{Shortcut}, 0} s_2 \xrightarrow{\text{Move}, +10} T$$

TD Update for s_0 :

$$V_2(s_0) = V_1(s_0) + \alpha(R_{t+1} + \gamma V_1(s_2) - V_1(s_0)) = -0.5 + 0.5[0 + 0.9(0) - (-0.5)] = -0.25$$

TD Update for s_2 :

$$V_2(s_2) = V_1(s_2) + \alpha(R_{t+1} + \gamma V_1(T) - V_1(s_2)) = 0 + 0.5[10 + 0.9(0) - 0] = 5$$

Observation

The shortcut state s_2 now looks promising because this sampled episode ended well.



Value Estimates Across Episodes

After	$V(s_0)$	$V(s_1)$	$V(s_2)$
Initial	0	0	0
Episode 1	-0.50	3.00	0
Episode 2	-0.25	3.00	5.00
Episode 3	0.725	4.50	5.00



Value Estimates Across Episodes

After	$V(s_0)$	$V(s_1)$	$V(s_2)$
Initial	0	0	0
Episode 1	-0.50	3.00	0
Episode 2	-0.25	3.00	5.00
Episode 3	0.725	4.50	5.00

Observations:

- ▶ TD learns from immediate rewards.
- ▶ TD also uses the current value of the next state.
- ▶ Values change as new sampled transitions are observed.



Advantages of TD Evaluation over MC Evaluation

- ▶ TD state evaluation can happen after every step while MC must wait until end of episode



Advantages of TD Evaluation over MC Evaluation

- ▶ TD state evaluation can happen after every step while MC must wait until end of episode
- ▶ TD learning can happen with incomplete sequences while MC cannot



Advantages of TD Evaluation over MC Evaluation

- ▶ TD state evaluation can happen after every step while MC must wait until end of episode
- ▶ TD learning can happen with incomplete sequences while MC cannot
- ▶ TD works for both episodic and continuing tasks while MC works only for episodic tasks



Advantages of TD Evaluation over MC Evaluation

- ▶ TD state evaluation can happen after every step while MC must wait until end of episode
- ▶ TD learning can happen with incomplete sequences while MC cannot
- ▶ TD works for both episodic and continuing tasks while MC works only for episodic tasks
- ▶ TD methods generally converge to optimality faster compared to MC methods



Advantages of TD Evaluation over MC Evaluation

- ▶ TD state evaluation can happen after every step while MC must wait until end of episode
- ▶ TD learning can happen with incomplete sequences while MC cannot
- ▶ TD works for both episodic and continuing tasks while MC works only for episodic tasks
- ▶ TD methods generally converge to optimality faster compared to MC methods
- ▶ TD methods have low variance and some bias while MC has high variance and zero bias



On-Policy TD Control: SARSA

TD Control: SARSA

- ▶ Like MC control, TD control is also based on action values
- ▶ SARSA is on-policy TD control i.e., target policy and behaviour policy are same
- ▶ Like in On-policy MC control, here also ϵ -greedy policy is used



TD Control: SARSA

- ▶ Like MC control, TD control is also based on action values
- ▶ SARSA is on-policy TD control i.e., target policy and behaviour policy are same
- ▶ Like in On-policy MC control, here also ϵ -greedy policy is used
- ▶ Update formula for estimating action values uses:
 - Present state S_t
 - Present action A_t
 - Observed reward R_{t+1}
 - Next state S_{t+1}
 - Next action A_{t+1}
- ▶ Update rule: $Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha[R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)]$



SARSA Algorithm

- ▶ **Input:** ϵ -greedy policy with small $\epsilon > 0$ and $\alpha \in (0, 1]$
- ▶ **Initialise:** $Q(s, a) \forall s \in \mathcal{S}, a \in \mathcal{A}(s)$ arbitrarily
- ▶ **Loop for each timestep t :**
 - Based on state S_t , take action A_t given by policy π based on current Q estimates
 - Observe R_{t+1} and S_{t+1}
 - Update Q-value of (S_t, A_t) : $Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha[R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)]$
- ▶ As Q-values get updated, the greedy action at each state might get updated until the optimal action in each state is identified



SARSA Algorithm

- ▶ **Input:** ϵ —greedy policy with small $\epsilon > 0$ and $\alpha \in (0, 1]$
- ▶ **Initialise:** $Q(s, a) \forall s \in \mathcal{S}, a \in \mathcal{A}(s)$ arbitrarily
- ▶ **Loop for each timestep t :**
 - Based on state S_t , take action A_t given by policy π based on current Q estimates
 - Observe R_{t+1} and S_{t+1}
 - Update Q-value of (S_t, A_t) : $Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha[R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)]$
- ▶ As Q-values get updated, the greedy action at each state might get updated until the optimal action in each state is identified

Result: SARSA algorithm converges to an optimal policy as long as all possible state-action pairs are visited sufficient number of times



Off-Policy TD Control: Q-Learning

- ▶ Q-learning is off-policy TD control i.e., target policy and behaviour policy are different
- ▶ ϵ -greedy policy is used to sample experience while greedy actions are used in the update rule



- ▶ Q-learning is off-policy TD control i.e., target policy and behaviour policy are different
- ▶ ϵ -greedy policy is used to sample experience while greedy actions are used in the update rule
- ▶ **Update rule:** $Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha[R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t)]$
- ▶ Q-learning is one of the most popular classical algorithms in RL developed by Watkins in 1989



Q-Learning Algorithm

- ▶ **Input:** ϵ -greedy policy with small $\epsilon > 0$ and $\alpha \in (0, 1]$
- ▶ **Initialise:** $Q(s, a) \forall s \in \mathcal{S}, a \in \mathcal{A}(s)$ arbitrarily
- ▶ **Loop for each timestep t :**
 - Based on state S_t , take action A_t given by policy π based on current Q estimates
 - Observe R_{t+1} and S_{t+1}
 - Update Q-value of $Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha[R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t)]$
- ▶ As Q-values get updated, the greedy action at each state might get updated until the optimal action in each state is identified



Q-Learning Algorithm

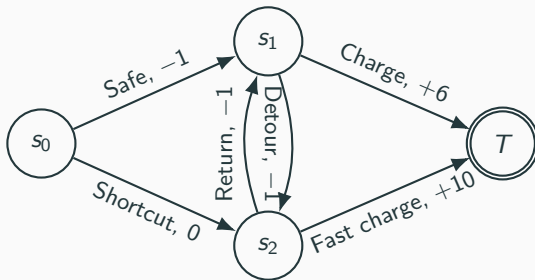
- ▶ **Input:** ϵ -greedy policy with small $\epsilon > 0$ and $\alpha \in (0, 1]$
- ▶ **Initialise:** $Q(s, a) \forall s \in \mathcal{S}, a \in \mathcal{A}(s)$ arbitrarily
- ▶ **Loop for each timestep t :**
 - Based on state S_t , take action A_t given by policy π based on current Q estimates
 - Observe R_{t+1} and S_{t+1}
 - Update Q-value of $Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha[R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t)]$
- ▶ As Q-values get updated, the greedy action at each state might get updated until the optimal action in each state is identified

Result: Q-learning algorithm converges to an optimal policy as long as all possible state-action pairs are visited sufficient number of times



Robot Example for SARSA and Q-Learning

- ▶ A robot wants to reach the charging station T .
- ▶ At s_0 , it can choose a safe path or a shortcut.
- ▶ At later states, it still has choices.
- ▶ This allows us to see the role of $\max_{a'} Q(s', a')$ clearly.



$$\gamma = 0.9, \quad \alpha = 0.5$$



Current Q-Value Estimates

Suppose the current action-value estimates are:

State	Action	Current Q-value
s_0	Safe	0.0
s_0	Shortcut	1.0
s_1	Charge	5.0
s_1	Detour	1.0
s_2	Fast charge	4.0
s_2	Return	2.0

Greedy Actions

At s_1 , the greedy action is **Charge**, because $Q(s_1, \text{Charge}) = 5.0$ is greater than $Q(s_1, \text{Detour}) = 1.0$.



Observed Transition Under an Exploratory Policy

Suppose the robot follows an ϵ -greedy behaviour policy.

And it observes the first transition as:

$$s_0 \xrightarrow{\text{Safe}, -1} s_1$$

At s_1 , suppose the behaviour policy chooses the exploratory action: **Detour** even though the greedy action at s_1 is **Charge**.

Important Point:

SARSA would use the actually chosen next action, **Detour** to update Q-value.

Q-learning ignores that next action in the target and uses the best next action, **Charge** to update the Q-value.



Q-Learning Update for $Q(s_0, \text{Safe})$

Observed transition:

$$s_0 \xrightarrow{\text{Safe}, -1} s_1$$

Current values at s_1 :

$$Q(s_1, \text{Charge}) = 5.0, \quad Q(s_1, \text{Detour}) = 1.0, \quad \max_{a'} Q(s_1, a') = 5.0$$

If SARSA were used, the target would be:

$$-1 + 0.9Q(s_1, \text{Detour}) = -1 + 0.9(1.0) = -0.1$$

But Q-learning uses:

$$-1 + 0.9 \max_{a'} Q(s_1, a') = -1 + 0.9(5.0) = 3.5$$

Core Difference

- ▶ SARSA learns the value of the behaviour actually followed.
- ▶ Q-learning learns toward the greedy policy, even while the behaviour policy explores.



A Second Q-Learning Update

Now, the robot actually takes the exploratory action:

$$s_1 \xrightarrow{\text{Detour}, -1} s_2$$

Current values at s_2 :

$$Q(s_2, \text{Fast charge}) = 4.0, \quad Q(s_2, \text{Return}) = 2.0, \quad \max_{a'} Q(s_2, a') = 4.0$$

Current value:

$$Q_{old}(s_1, \text{Detour}) = 1.0$$

Q-learning target:

$$-1 + 0.9(4.0) = 2.6$$

Q-Learning Update:

$$Q_{new}(s_1, \text{Detour}) = 1.0 + 0.5[2.6 - 1.0] = 1.8$$



Q-Values Before and After Updates

State	Action	Before	After
s_0	Safe	0.0	1.75
s_0	Shortcut	1.0	1.0
s_1	Charge	5.0	5.0
s_1	Detour	1.0	1.8
s_2	Fast charge	4.0	4.0
s_2	Return	2.0	2.0

Key Takeaway:

In Q-learning, experience may be exploratory but update is greedy, making it off-policy learning.



Cliff Walking Example: SARSA Vs Q-Learning

- ▶ **Goal:** To go from initial state to goal state without passing through the cliff area and in minimum number of steps
- ▶ **Environment:** Grid environment with multiple blocks as shown



Cliff Walking Example: SARSA Vs Q-Learning

- ▶ **Goal:** To go from initial state to goal state without passing through the cliff area and in minimum number of steps
- ▶ **Environment:** Grid environment with multiple blocks as shown



Figure 1: Cliff World Environment



Cliff Walking Example: SARSA Vs Q-Learning

- ▶ **Goal:** To go from initial state to goal state without passing through the cliff area and in minimum number of steps
- ▶ **Environment:** Grid environment with multiple blocks as shown
- ▶ **Rules:**
 - Move only from one block to an adjacent block in a step
 - Allowed to move in any direction except the diagonal
 - If you step in blue coloured blocks, you fall off the cliff and have to re-start



Figure 1: Cliff World Environment



Cliff Walking Example: SARSA Vs Q-Learning

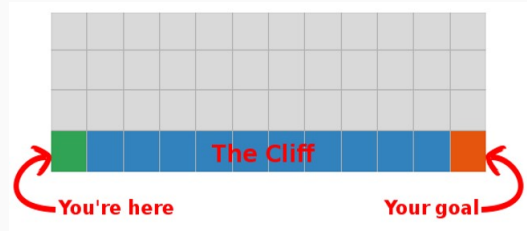


Figure 2: Cliff World Environment



Cliff Walking Example: SARSA Vs Q-Learning

► States and Episodes:

- Each block represents a state of the environment
- Initial state and goal state are fixed
- Episode ends when one reaches goal state or a cliff block



Figure 2: Cliff World Environment



Cliff Walking Example: SARSA Vs Q-Learning

► States and Episodes:

- Each block represents a state of the environment
- Initial state and goal state are fixed
- Episode ends when one reaches goal state or a cliff block

► Actions: Left, Right, Top, Down



Figure 2: Cliff World Environment



Cliff Walking Example: SARSA Vs Q-Learning

► States and Episodes:

- Each block represents a state of the environment
- Initial state and goal state are fixed
- Episode ends when one reaches goal state or a cliff block

► Actions: Left, Right, Top, Down

► Reward Function:

- Goal State : 0
- Any Cliff State: -100
- Any other state: -1



Figure 2: Cliff World Environment



Cliff Walking Example: SARSA Vs Q-Learning

► States and Episodes:

- Each block represents a state of the environment
- Initial state and goal state are fixed
- Episode ends when one reaches goal state or a cliff block

► Actions: Left, Right, Top, Down

► Reward Function:

- Goal State : 0
- Any Cliff State: -100
- Any other state: -1

Assignment: Write a python code to simulate the environment and find an optimal policy using SARSA and Q-Learning methods



Figure 2: Cliff World Environment



Expected SARSA

- ▶ On-policy approach which is an extension to with a minor change in the TD target
- ▶ TD target takes into account all possible actions in next state instead of a random action like in SARSA



Expected SARSA

- ▶ On-policy approach which is an extension to with a minor change in the TD target
- ▶ TD target takes into account all possible actions in next state instead of a random action like in SARSA
- ▶ **Approach:** Uses the expected Q-value based on the probability of taking each action under current policy
- ▶ **Update rule:**

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha[R_{t+1} + \gamma \mathbb{E}_{\pi}[Q(S_{t+1}, A_{t+1})|S_{t+1}] - Q(S_t, A_t)]$$
$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha[R_{t+1} + \gamma \sum_a \pi(a|S_{t+1})Q(S_{t+1}, a) - Q(S_t, A_t)]$$



Expected SARSA

- ▶ On-policy approach which is an extension to with a minor change in the TD target
- ▶ TD target takes into account all possible actions in next state instead of a random action like in SARSA
- ▶ **Approach:** Uses the expected Q-value based on the probability of taking each action under current policy
- ▶ **Update rule:**

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha[R_{t+1} + \gamma \mathbb{E}_{\pi}[Q(S_{t+1}, A_{t+1})|S_{t+1}] - Q(S_t, A_t)]$$
$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha[R_{t+1} + \gamma \sum_a \pi(a|S_{t+1})Q(S_{t+1}, a) - Q(S_t, A_t)]$$

- ▶ **Advantage:** Reduces variance in updates resulting in more stability and better convergence properties



Summary of TD Control Methods

- ▶ All TD control algorithms use an update rule to update the Q-values of a policy
- ▶ Optimal policy is to take greedy actions i.e., take actions with highest Q-value in a given state
- ▶ SARSA's update rule is based on a random action taken in next state

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha[R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)]$$

- ▶ Q-learning's update rule is based on Q-value of greedy action taken in next state

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha[R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t)]$$

- ▶ Expected SARSA's update rule is based on a average of Q-values of all possible actions in next state

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha[R_{t+1} + \gamma \sum_a \pi(a|S_{t+1})Q(S_{t+1}, a) - Q(S_t, A_t)]$$

- ▶ All of them have proven convergence properties under reasonable assumptions



n-Step TD Methods

n-step TD methods

- ▶ Generalisation of MC methods and 1-step TD methods
- ▶ **Approach:** Bootstrapping is done and values are updated after n-steps



n-step TD methods

- Generalisation of MC methods and 1-step TD methods
- **Approach:** Bootstrapping is done and values are updated after n-steps

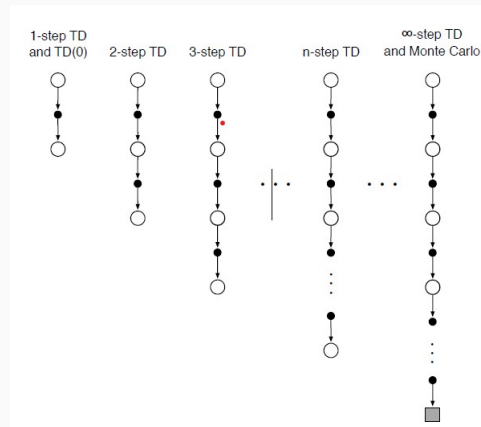


Figure 3: Backup diagrams of n-step TD methods



n-step TD methods

- Generalisation of MC methods and 1-step TD methods
- **Approach:** Bootstrapping is done and values are updated after n-steps
- Advantages over MC and 1-step TD methods:
 - n-step TD gives a better estimate of the TD target than 1-step TD
 - Lower bias than TD(0) and lower variance than MC

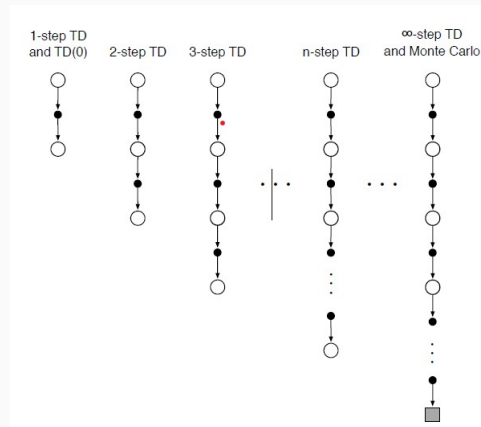


Figure 3: Backup diagrams of n-step TD methods



n-step TD methods

- Generalisation of MC methods and 1-step TD methods
- **Approach:** Bootstrapping is done and values are updated after n -steps
- Advantages over MC and 1-step TD methods:
 - n -step TD gives a better estimate of the TD target than 1-step TD
 - Lower bias than TD(0) and lower variance than MC
 - Faster learning than MC and more stable than TD(0)
 - Flexibility to tune the value of n based on the problem

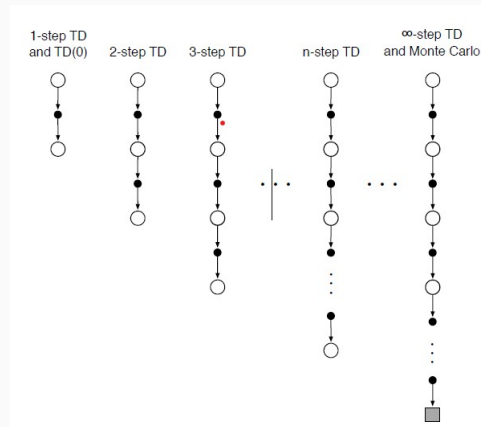


Figure 3: Backup diagrams of n -step TD methods



- ▶ **Task Type:** Moderate values of n for continuing tasks to balance bias and variance



Factors affecting choice of n

- ▶ **Task Type:** Moderate values of n for continuing tasks to balance bias and variance
- ▶ **Rewards:** Larger values of n in case of delayed rewards and vice versa



Factors affecting choice of n

- ▶ **Task Type:** Moderate values of n for continuing tasks to balance bias and variance
- ▶ **Rewards:** Larger values of n in case of delayed rewards and vice versa
- ▶ **Type of Environment:** Smaller value of n for highly stochastic environments to control variance



Factors affecting choice of n

- ▶ **Task Type:** Moderate values of n for continuing tasks to balance bias and variance
- ▶ **Rewards:** Larger values of n in case of delayed rewards and vice versa
- ▶ **Type of Environment:** Smaller value of n for highly stochastic environments to control variance
- ▶ **Memory and Computation:** Larger values of n require more memory and high computation power



Factors affecting choice of n

- ▶ **Task Type:** Moderate values of n for continuing tasks to balance bias and variance
- ▶ **Rewards:** Larger values of n in case of delayed rewards and vice versa
- ▶ **Type of Environment:** Smaller value of n for highly stochastic environments to control variance
- ▶ **Memory and Computation:** Larger values of n require more memory and high computation power
- ▶ **Learning rate:** Learning rate α and n should adjusted inversely for stability and convergence



Factors affecting choice of n

- ▶ **Task Type:** Moderate values of n for continuing tasks to balance bias and variance
- ▶ **Rewards:** Larger values of n in case of delayed rewards and vice versa
- ▶ **Type of Environment:** Smaller value of n for highly stochastic environments to control variance
- ▶ **Memory and Computation:** Larger values of n require more memory and high computation power
- ▶ **Learning rate:** Learning rate α and n should adjusted inversely for stability and convergence
- ▶ **Practice:** Tuning of n through experimentation and dynamic adjustment are recommended in practice to arrive at a suitable value



- ▶ TD target for an n-step state value update:

$$G_{t:t+n} = R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^{n-1} R_{t+n} + \gamma^n V(S_{t+n})$$

where $V(S_{t+n})$ is the current estimate of the value of the state visited at time $(t + n)$

- ▶ Update rule:

$$V(S_t) \leftarrow V(S_t) + \alpha [G_{t:t+n} - V(S_t)]$$



- ▶ TD target for an n-step state value update:

$$G_{t:t+n} = R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^{n-1} R_{t+n} + \gamma^n V(S_{t+n})$$

where $V(S_{t+n})$ is the current estimate of the value of the state visited at time $(t + n)$

- ▶ Update rule:

$$V(S_t) \leftarrow V(S_t) + \alpha[G_{t:t+n} - V(S_t)]$$

- ▶ **Notes:**

- No updates are made during the first $(n - 1)$ steps of an episode
- If $(t + n) > T$ i.e., episode terminated before n -steps from state t , then missing rewards are taken as 0



n-step TD Control: SARSA Algorithm

- ▶ **Input:** ϵ -greedy policy with small $\epsilon > 0$ and $\alpha \in (0, 1]$
- ▶ **Initialise:** $Q(s, a) \forall s \in \mathcal{S}, a \in \mathcal{A}(s)$ arbitrarily
- ▶ **Loop for each timestep t :**
 - Sample a trajectory starting with S_t until S_{t+n} or S_T taking actions $A_t, A_{t+1}, \dots$ given by policy π based on current Q estimates
 - If $(t+n) > T$, $G_{t:t+n} = \sum_{i=t+1}^T \gamma^{i-t-1} R_i$
 - else: $G_{t:t+n} = \sum_{i=t+1}^T \gamma^{i-t-1} R_i + Q(S_{t+n}, A_{t+n})$
 - Update Q-value of (S_t, A_t) : $Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha[G_{t:t+n} - Q(S_t, A_t)]$

Note: Convergence properties of this algorithm are similar to other TD control algorithms



TD(λ)

Composite Return as Target

- **Recall:** n -step return is the sum of the first n rewards and the value of the state reached in n steps

$$G_{t:t+n} = R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^{n-1} R_{t+n} + \gamma^n V(S_{t+n})$$



Composite Return as Target

- **Recall:** n -step return is the sum of the first n rewards and the value of the state reached in n steps

$$G_{t:t+n} = R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^{n-1} R_{t+n} + \gamma^n V(S_{t+n})$$

- **Q:** Can we combine multiple n -step returns with different values of n to get a target?



Composite Return as Target

- **Recall:** n -step return is the sum of the first n rewards and the value of the state reached in n steps

$$G_{t:t+n} = R_{t+1} + \gamma R_{t+2} + \dots + \gamma^{n-1} R_{t+n} + \gamma^n V(S_{t+n})$$

- **Q:** Can we combine multiple n -step returns with different values of n to get a target?
- **A:** Yes! Multiple n -step returns can be weighed and added to get a composite return as target



Composite Return as Target

- **Recall:** n -step return is the sum of the first n rewards and the value of the state reached in n steps

$$G_{t:t+n} = R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^{n-1} R_{t+n} + \gamma^n V(S_{t+n})$$

- **Q:** Can we combine multiple n -step returns with different values of n to get a target?
- **A:** Yes! Multiple n -step returns can be weighed and added to get a composite return as target
- **Condition:** Weights should be positive and add to 1



Composite Return as Target

- **Recall:** n -step return is the sum of the first n rewards and the value of the state reached in n steps

$$G_{t:t+n} = R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^{n-1} R_{t+n} + \gamma^n V(S_{t+n})$$

- **Q:** Can we combine multiple n -step returns with different values of n to get a target?
- **A:** Yes! Multiple n -step returns can be weighed and added to get a composite return as target
- **Condition:** Weights should be positive and add to 1
- **Example:** Composite return of half of 2-step return and half of 4-step return can be the target: $\frac{1}{2} G_{t:t+2} + \frac{1}{2} G_{t:t+4}$

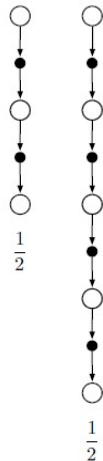


Composite Return as Target

- **Recall:** n -step return is the sum of the first n rewards and the value of the state reached in n steps

$$G_{t:t+n} = R_{t+1} + \gamma R_{t+2} + \dots + \gamma^{n-1} R_{t+n} + \gamma^n V(S_{t+n})$$

- **Q:** Can we combine multiple n -step returns with different values of n to get a target?
- **A:** Yes! Multiple n -step returns can be weighed and added to get a composite return as target
- **Condition:** Weights should be positive and add to 1
- **Example:** Composite return of half of 2-step return and half of 4-step return can be the target: $\frac{1}{2} G_{t:t+2} + \frac{1}{2} G_{t:t+4}$
- **Advantage:** A wide of range of algorithms can be developed based on this idea



λ -return

- ▶ λ -return is a particular way of averaging multiple n -step returns
- ▶ In TD(λ) methods, λ -return becomes the TD target



λ -return

- ▶ λ -return is a particular way of averaging multiple n -step returns
- ▶ In TD(λ) methods, λ -return becomes the TD target
- ▶ **Idea:** Average all n -step returns with exponentially decaying weights controlled by λ

$$G_t^\lambda = (1 - \lambda) \sum_{n=1}^{\infty} \lambda^{n-1} G_{t:t+n}$$

$$G_t^\lambda = (1 - \lambda)G_{t:t+1} + (1 - \lambda)\lambda G_{t:t+2} + (1 - \lambda)\lambda^2 G_{t:t+3} + \dots$$

- ▶ Weight fades by λ with each additional step
- ▶ λ controls the trade-off between importance of immediate and future rewards



- ▶ λ -return is a particular way of averaging multiple n -step returns
- ▶ In TD(λ) methods, λ -return becomes the TD target
- ▶ **Idea:** Average all n -step returns with exponentially decaying weights controlled by λ

$$G_t^\lambda = (1 - \lambda) \sum_{n=1}^{\infty} \lambda^{n-1} G_{t:t+n}$$

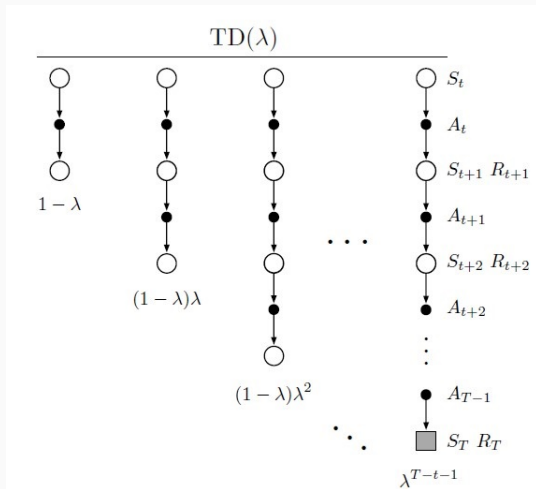
$$G_t^\lambda = (1 - \lambda)G_{t:t+1} + (1 - \lambda)\lambda G_{t:t+2} + (1 - \lambda)\lambda^2 G_{t:t+3} + \dots$$

- ▶ Weight fades by λ with each additional step
- ▶ λ controls the trade-off between importance of immediate and future rewards
- ▶ **Notes:**
 - At terminal state, all subsequent n -step returns are equal to conventional return G_t
 - If $\lambda = 0$, then it becomes 1-step TD update or TD(0)
 - If $\lambda = 1$, then it becomes MC update



Backup diagram of TD(λ)

Referred to as forward view TD(λ) because it looks into the future to compute G_t^λ



Issues with Forward View TD(λ)

- Issues forward view TD(λ):



- Issues forward view TD(λ):
 - Like in MC, λ -return can only be computed for completed episodes
 - Computing n -step returns for every value of n and for each state or state-action pair is complex and inefficient



Issues with Forward View TD(λ)

- ▶ Issues forward view TD(λ):
 - Like in MC, λ -return can only be computed for completed episodes
 - Computing n -step returns for every value of n and for each state or state-action pair is complex and inefficient
- ▶ Q: How to overcome the above issues?
- ▶ Let us understand the problem of Credit Assignment in RL!



- **Recall:** In sequential decision making, present actions impact present as well as future states and rewards



Credit Assignment Problem

- ▶ **Recall:** In sequential decision making, present actions impact present as well as future states and rewards
- ▶ **Credit Assignment:**
 - Determining which actions taken in the past are responsible for the rewards received in the future
 - Agent needs to figure out which of its previous actions contributed to a reward and to what extent



Credit Assignment Problem

- ▶ **Recall:** In sequential decision making, present actions impact present as well as future states and rewards
- ▶ **Credit Assignment:**
 - Determining which actions taken in the past are responsible for the rewards received in the future
 - Agent needs to figure out which of its previous actions contributed to a reward and to what extent
- ▶ Credit assignment is analogous to determining loss contributing factors during back propagation in neural networks
- ▶ Most RL algorithms assume that present action impacts all future rewards though more weightage is given to immediate rewards (discounting)



Solving the Credit Assignment Problem

► MC Methods

- MC methods wait until end of episode to assign credit based on accumulated return
- Each state-action pair is credited based on the complete return from that point to the end of the episode



Solving the Credit Assignment Problem

► MC Methods

- MC methods wait until end of episode to assign credit based on accumulated return
- Each state-action pair is credited based on the complete return from that point to the end of the episode

► TD Methods

- Assigns credit faster than Monte Carlo methods
- Updates value estimates incrementally at each time step based on the TD error



Solving the Credit Assignment Problem

► MC Methods

- MC methods wait until end of episode to assign credit based on accumulated return
- Each state-action pair is credited based on the complete return from that point to the end of the episode

► TD Methods

- Assigns credit faster than Monte Carlo methods
- Updates value estimates incrementally at each time step based on the TD error

► Q: Is there an efficient way to assign credit in $TD(\lambda)$ methods?



Solving the Credit Assignment Problem

► MC Methods

- MC methods wait until end of episode to assign credit based on accumulated return
- Each state-action pair is credited based on the complete return from that point to the end of the episode

► TD Methods

- Assigns credit faster than Monte Carlo methods
- Updates value estimates incrementally at each time step based on the TD error

► Q: Is there an efficient way to assign credit in $TD(\lambda)$ methods?

► A: Eligibility traces to the rescue!



- Eligibility traces are an efficient way to solving credit assignment problem and an efficient alternative to λ -return



- ▶ Eligibility traces are an efficient way to solving credit assignment problem and an efficient alternative to λ -return
- ▶ Approach:
 - Keep track of the "eligibility" of each state s or state-action pair (s, a) for receiving value updates based on future rewards
 - Facilitate temporal credit assignment by exponentially decaying eligibility over time i.e., recent actions have more influence on current reward
 - Much simpler approach compared to maintaining multiple returns



- ▶ Eligibility traces are an efficient way to solving credit assignment problem and an efficient alternative to λ -return
- ▶ Approach:
 - Keep track of the "eligibility" of each state s or state-action pair (s, a) for receiving value updates based on future rewards
 - Facilitate temporal credit assignment by exponentially decaying eligibility over time i.e., recent actions have more influence on current reward
 - Much simpler approach compared to maintaining multiple returns
- ▶ Referred to as backward view $TD(\lambda)$ because the influence of the TD error is propagated backward to all states that have been visited



Backward view TD(λ)

► Eligibility Traces:

- Traces keep track of how recently and frequently each state-action pair has been visited

$$E_0(s, a) = 0 \quad \forall s, a$$

$$E_t(s, a) = \gamma\lambda E_{t-1}(s, a) + \mathbf{1}(S_t = s, A_t = a)$$

- $\mathbf{1}(S_t = s, A_t = a)$ is an indicator function
- Eligibility trace of a state-action pair will tend to 0 if a state-action pair is not visited again in a sample trajectory



Backward view TD(λ)

► Eligibility Traces:

- Traces keep track of how recently and frequently each state-action pair has been visited

$$E_0(s, a) = 0 \quad \forall s, a$$

$$E_t(s, a) = \gamma \lambda E_{t-1}(s, a) + \mathbf{1}(S_t = s, A_t = a)$$

- $\mathbf{1}(S_t = s, A_t = a)$ is an indicator function
 - Eligibility trace of a state-action pair will tend to 0 if a state-action pair is not visited again in a sample trajectory
- λ : Trace-Decay Parameter controls the rate at which credit decays over time.
- In other words, it determines how much responsibility for a future error is assigned to past state-action pairs.
- λ is like a memory decay factor. If a state-action pair is not re-visited, the memory about it fades
- γ is the discounting factor which controls the present value of future rewards (analogous to time value of money)



Backward view TD(λ)

- ▶ TD Error: $\delta_t = R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)$



Backward view TD(λ)

- ▶ **TD Error:** $\delta_t = R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)$
- ▶ **Q-Value Update Rule:** TD error is weighted by the eligibility trace during update of Q-value

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \delta_t E_t(s, a)$$

- ▶ **Example:**
 - Consider update of (S_k, A_k) that was visited k steps ago based on TD error at present time t : δ_t
 - Value information in δ_t is $t - k$ steps in future from perspective of state-action pair S_k, A_k and must be discounted by γ^{t-k}
 - Credit for this error should fade which happens through λ^{t-k}
- ▶ Therefore, eligibility trace formula elegantly combined both these needs while updating the value of a state-action pair
- ▶ **Note:** At every timestep, all the Q-values are updated using the update rule



- ▶ n-step return in TD learning can be replaced by composite return to generalise TD learning further



Summary

- ▶ n -step return in TD learning can be replaced by composite return to generalise TD learning further
- ▶ λ -return averages over all possible n -step returns weighing them appropriately



- ▶ n-step return in TD learning can be replaced by composite return to generalise TD learning further
- ▶ λ -return averages over all possible n-step returns weighing them appropriately
- ▶ Forward view $TD(\lambda)$ uses λ -return as TD target to learn an optimal policy



- ▶ n -step return in TD learning can be replaced by composite return to generalise TD learning further
- ▶ λ -return averages over all possible n -step returns weighing them appropriately
- ▶ Forward view $TD(\lambda)$ uses λ -return as TD target to learn an optimal policy
- ▶ Backward view $TD(\lambda)$ overcomes the issues in computing λ -return by resolving the credit assignment problem using eligibility traces



- ▶ n -step return in TD learning can be replaced by composite return to generalise TD learning further
- ▶ λ -return averages over all possible n -step returns weighing them appropriately
- ▶ Forward view $TD(\lambda)$ uses λ -return as TD target to learn an optimal policy
- ▶ Backward view $TD(\lambda)$ overcomes the issues in computing λ -return by resolving the credit assignment problem using eligibility traces
- ▶ Eligibility traces are an efficient way to trace the state or state-action pairs visited and update Q -values incrementally according to their eligibility

